

5 实验评估

本章围绕五个研究问题展开：RQ1，RiskBudget-SAC 在平稳负载下能否降低 aggregate SLA 违约率与 p99，并满足估计能耗预算；RQ2，面对 random-switch 到达变化时，方法能否保持动态适应；RQ3，aggregate 改善是否以牺牲某一 DNN 为代价；RQ4，Historical Feedback GRU 与 Causal Risk Gate 如何共同影响结果；RQ5，Gate 的在线估计与激活行为是否符合设计，部署开销是否可接受。所有主结果均来自同一个冻结检查点，结论严格限定在当前评估协议。

5.1 实验设置

5.1.1 硬件 Profile 与 DNN 工作负载

系统包含 RTX 3060 Laptop GPU 与 Apple M3 两类执行设备，分别通过 CUDA 和 MPS 运行 MobileNetV3-Small、EfficientNet-B0 与 ResNet50。对每个 DNN-设备-batch 组合采集基准执行时延、吞吐和单位请求执行能耗，并将其作为第三章式 (7) 的基准服务特性。允许 batch size 为 {1,2,4,8,16,32}。M3 的 18 个组合均具有有效 profile；RTX 3060 有 17 个有效组合，ResNet50 的 batch=1 测量不稳定，环境按既定 padding/bucketing 规则映射到最小有效 batch=2。该替代规则对所有方法一致。

表 5.1 硬件与 Profile 概要

项目	设置
执行设备	RTX 3060 Laptop GPU (CUDA); Apple M3 (MPS)
DNN	MobileNetV3-Small; EfficientNet-B0; ResNet50
Batch size	1, 2, 4, 8, 16, 32
设备并发槽位	RTX 3060: 2; M3: 1
Profile 指标	执行时延、吞吐、单位请求执行能耗
有效组合	RTX 3060: 17/18; M3: 18/18

运行时共置干扰采用有界随机减速代理，并随 DNN 组合和 batch 变化。独立硬件测量已经确认共置干扰具有明显方向性，但主实验没有把完整方向查找表直接写入训练环境，因此本文将其定位为“由真实 profile 驱动、含随机运行偏移的仿真”，而不是完全由硬件干扰表校准的数字孪生。

5.1.2 事件驱动环境与负载场景

静态负载覆盖 $\rho \in \{0.40, 0.50, 0.60, 0.70, 0.75, 0.80, 0.85, 0.90\}$ 。动态训练与测试包含 random-switch 和 model-mix-drift；主文重点报告 random-switch，其中系统到达强度在 0.25 s 片段间切换，平均目标负载为 $\rho=0.70$ 。调度器不接收 ρ 或场景名称，只能观察队列、deadline、设备状态、已经发生的到达与已完成 batch 反馈。

deployment-aware 评价在 edge-side 完成时延上叠加动作无关上行：上行速率服从 $U(5,10)$ MB/s；无文件级输入时，请求数据大小服从 $U(50,100)$ KB。所有方法在相同条件、相同评估种子下共享相同输入大小与速率样本。由于上行与调度动作无关，它不改变策略训练和动作排序，但会消耗请求 SLA 裕量。结果回传因分类输出很小而忽略。

5.1.3 训练与评估协议

RiskBudget-SAC 从头训练 400 episodes，40% episode 采样八档静态负载，60% episode 采样动态场景；全程使用一个 Actor、一个 Twin Critic 组和一个 replay buffer，不切换检查点。训练随机种子为 0。主评估冻结同一个 UnifiedRiskBudget.pt，在每个条件下使用 5 个配对环境评估种子，运行 60 s，并将前 1 s 作为 warm-up。误差条为 5 个环境评估种子上的双侧 95% t 置信区间，不能解释为多次独立训练的方差。

表 5.2 训练与评估配置

类别	设置
训练	400 episodes; seed 0; 40% 静态 + 60% 动态
动作空间	37 个离散动作: WAIT 与 DNN-device-batch 组合
网络	History-GRU 64; Actor hidden 192; action embedding 96
优化	Actor/Critic learning rate $3e-4$; 自动熵温度
Replay	容量 90,000; batch 256; 场景/压力/tail 平衡采样
物理时间 n-step	窗口 0.05 s; 最多 64 个决策
SLA	每 DNN 独立对偶; 目标违约比例 0.02
能耗预算	0.109587 J/request (profile-estimated)
主评估	60 s; 1 s warm-up; 5 个 paired evaluation seeds

5.1.4 Baselines、指标与公平性

比较方法包括学习式 Coinf-Adapted，以及 TF-Serving-style、Triton-Dynamic-Batcher、Ebird-style、EDF-StaticBatch 和 FCFS。所有方法共享相同请求流、模型级 deadline、设备容量、profile、基础物理动作 mask、运行时扰动和上行样本。启发式方法不读取未来到达或隐藏干扰。Coinf-Adapted 保留 Coinf 的性能预测、紧急任务识别与 A2C 调度思想，并为当前双设备动作空间增加显式设备选择；因此它是面向当前环境的适配版本，而不是原论文二进制实现。

表 5.3 Baseline 定义与信息边界

方法	核心规则或学习机制	在线反馈
Coinf-Adapted	profile + 紧急度识别 + A2C 组批/路由	完成时延分位修正
TF-Serving-style	固定 batch cap=32 + 最大等待	队列/设备状态
Triton-Dynamic-Batcher	preferred batch {16,32} + queue delay	队列/设备状态
Ebird-style	多 batch 并发组合 {1,1,2,4,8,16}	队列/设备状态

方法	核心规则或学习机制	在线反馈
EDF-StaticBatch	最早 deadline + 固定 cap + profile 路由	队列/设备状态
FCFS	全局最早到达 + 固定 cap + 首个可用设备	队列/设备状态

主要指标为 deployment-aware 请求级 SLA 违约率、deployment-aware p99 完成时延、吞吐量和 profile-estimated 单位请求执行能耗。静态主表对八档 ρ 等权宏平均；动态表报告 random-switch 结果。本文的主要目标是控制 SLA 与 p99，同时检查能耗是否低于预算，而不是声称能耗全局最小。

5.2 总体性能

5.2.1 平稳负载

图 5.1 展示八档静态负载下的三项主要指标。随着 ρ 升高，各方法均面临更强队列压力，但不同调度方式在 tail latency 上迅速分化。表 5.4 给出八档条件等权宏平均。

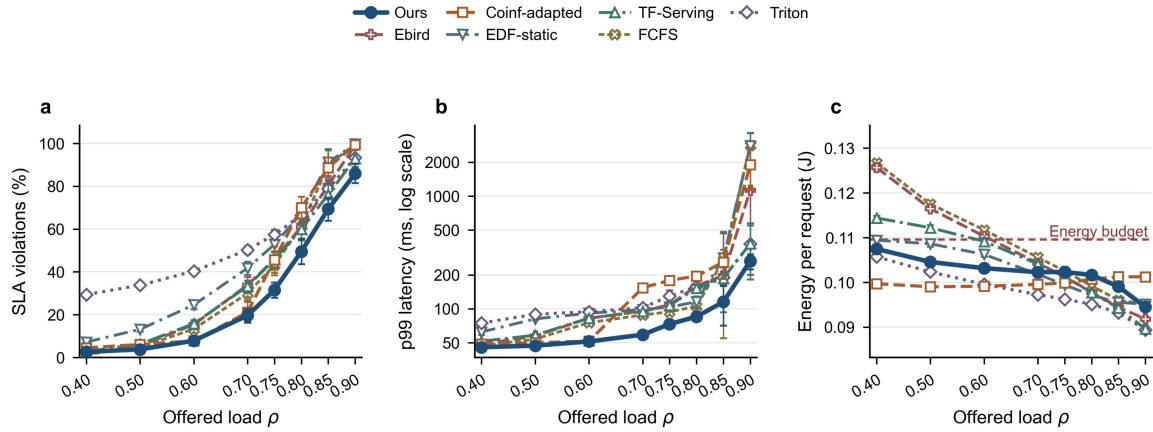


图 5.1 八档静态负载下各方法的 deployment-aware SLA 违约率、p99 时延与 profile-estimated 单位请求能耗。

点为 5 个 paired evaluation seeds 的均值，误差条为双侧 95% t 置信区间。

表 5.4 静态八档负载的宏平均结果

方法	SLA 违约率 (%)	p99 (ms)	能耗 (J/request)
RiskBudget-SAC	33.75	92.68	0.1019
Coinf-Adapted	42.89	354.70	0.1001
TF-Serving-style	41.58	139.18	0.1028
Triton-Dynamic-Batcher	56.22	151.75	0.0974
Ebird-style	42.84	234.94	0.1053
EDF-StaticBatch	49.70	452.80	0.1017
FCFS	42.92	445.17	0.1068

RiskBudget-SAC 的宏平均违约率为 33.75%，最强违约率基线 TF-Serving-style 为 41.58%，相对降低 18.8%。其 p99 为 92.68 ms，较最强 p99 基线的 139.18 ms 降低 33.4%。单位请求能耗为 0.1019 J，低于 0.109587 J 的预设预算，但并非最低：Triton-Dynamic-Batcher 为 0.0974 J。结果说

明该方法以有限的估计能耗开销换取更明显的 SLA 与 tail latency 改善，而不是获得所有指标的绝对最优。

5.2.2 Random-Switch 动态负载

random-switch 在短片段之间改变请求强度，要求策略根据已经发生的到达和完成反馈调整调度。图 5.2 与表 5.5 给出 60 s 结果。

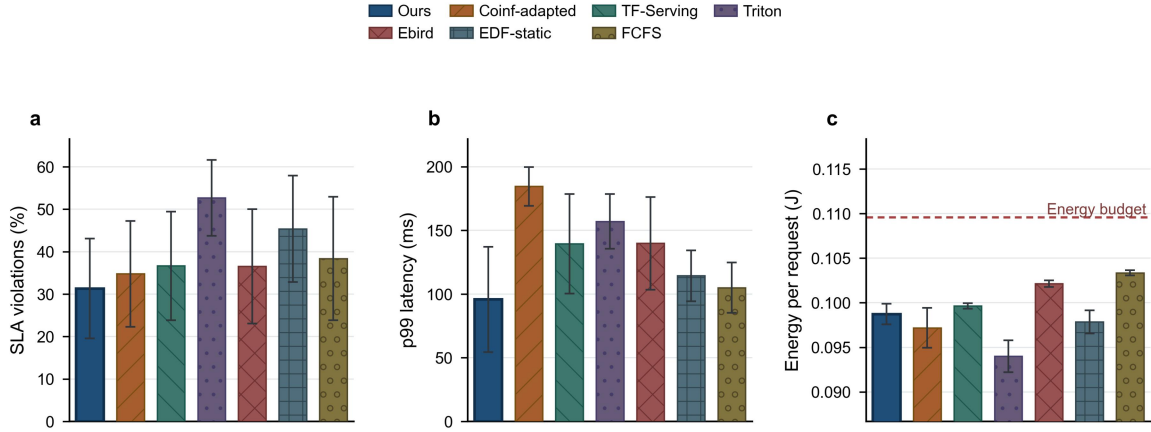


图 5.2 Random-switch 场景下各方法的 deployment-aware SLA 违约率、p99 时延与 profile-estimated 单位请求能耗；误差条口径同图 5.1。

表 5.5 Random-switch 场景结果

方法	SLA 违约率 (%)	p99 (ms)	能耗 (J/request)
RiskBudget-SAC	31.32	95.84	0.0987
Coinf-Adapted	34.80	184.54	0.0972
TF-Serving-style	36.66	139.44	0.0997
Triton-Dynamic-Batcher	52.68	157.10	0.0940
Ebird-style	36.56	139.91	0.1021
EDF-StaticBatch	45.39	114.36	0.0979
FCFS	38.38	105.01	0.1034

RiskBudget-SAC 的违约率为 31.32%，较最强违约率基线 Coinf-Adapted 的 34.80% 相对降低 10.0%；p99 为 95.84 ms，较最强 p99 基线 FCFS 的 105.01 ms 降低 8.7%。与学习式 Coinf-Adapted 相比，其 p99 从 184.54 ms 降至 95.84 ms。能耗 0.0987 J 仍低于预算，但高于 Triton 的 0.0940 J。动态结果的置信区间较宽，说明不同切换轨迹对 tail 指标影响明显，因此本节只主张平均改善，不声称每条轨迹均占优。

5.3 分 DNN 性能

aggregate 指标可能掩盖某个模型被长期牺牲。图 5.3 分别报告 MobileNetV3-Small、EfficientNet-B0 与 ResNet50 在静态和 random-switch 条件下的违约率与 p99。

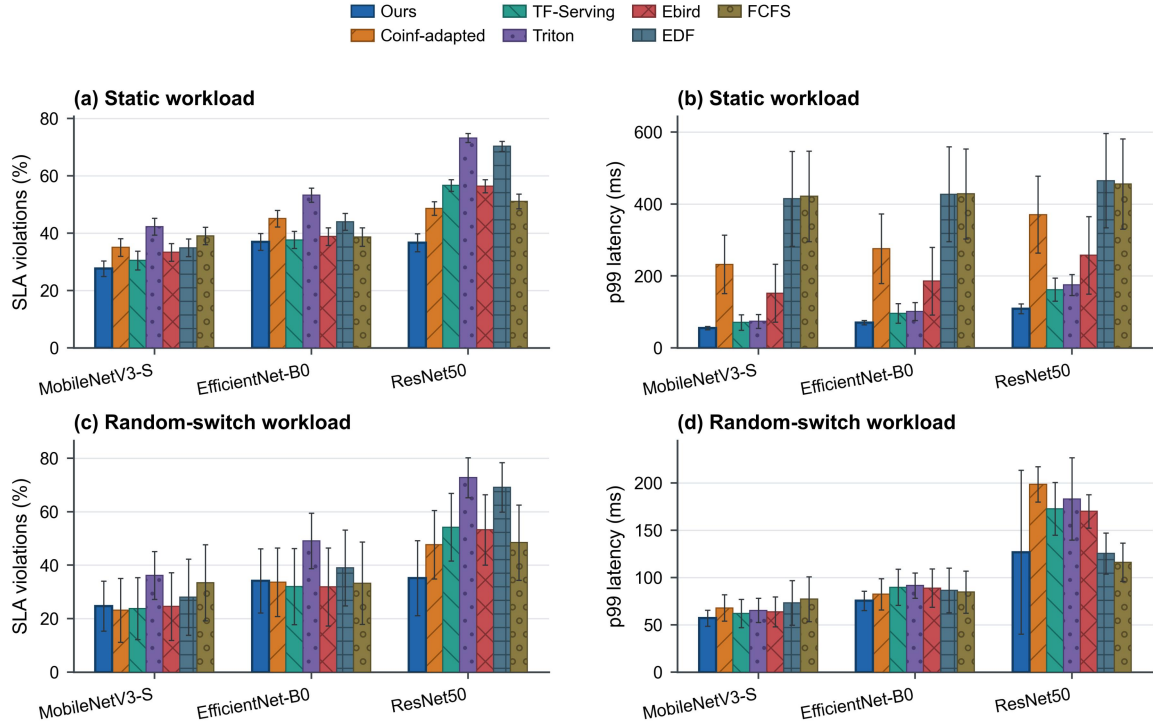


图 5.3 静态与 random-switch 场景的分 DNN SLA 违约率和 p99 时延，均基于 60 s、5 个 paired evaluation seeds。

表 5.6 RiskBudget-SAC 的分 DNN 结果

场景	DNN	SLA 违约率 (%)	p99 (ms)
静态宏平均	MobileNetV3-Small	27.64	55.20
静态宏平均	EfficientNet-B0	36.99	70.05
静态宏平均	ResNet50	36.67	108.98
Random-switch	MobileNetV3-Small	24.68	56.99
Random-switch	EfficientNet-B0	34.14	75.41
Random-switch	ResNet50	35.13	126.79

三种 DNN 均得到持续服务，没有出现某一模型完全饥饿。MobileNetV3-Small 的违约率和 p99 最低，ResNet50 的执行开销较大，因而 p99 最高。需要谨慎解释的是，RiskBudget-SAC 并未在每个模型的每项指标上都最好：random-switch 下 ResNet50 的 p99 高于 FCFS，并略高于 EDF-StaticBatch；其优势更多体现在较低的 ResNet50 违约率和整体风险平衡。因此，aggregate 改善不是简单忽略某一 DNN，但模型间公平性仍有进一步优化空间。

5.4 Gate 在动态负载下的行为

为验证 Gate 是否按设计响应已经揭示的负载变化，我们记录 60 s random-switch 轨迹中的真实到达率（仅用于离线诊断）、EWMA 估计、队列长度与 Gate 激活率。图 5.4 汇总 5 个评估种子。

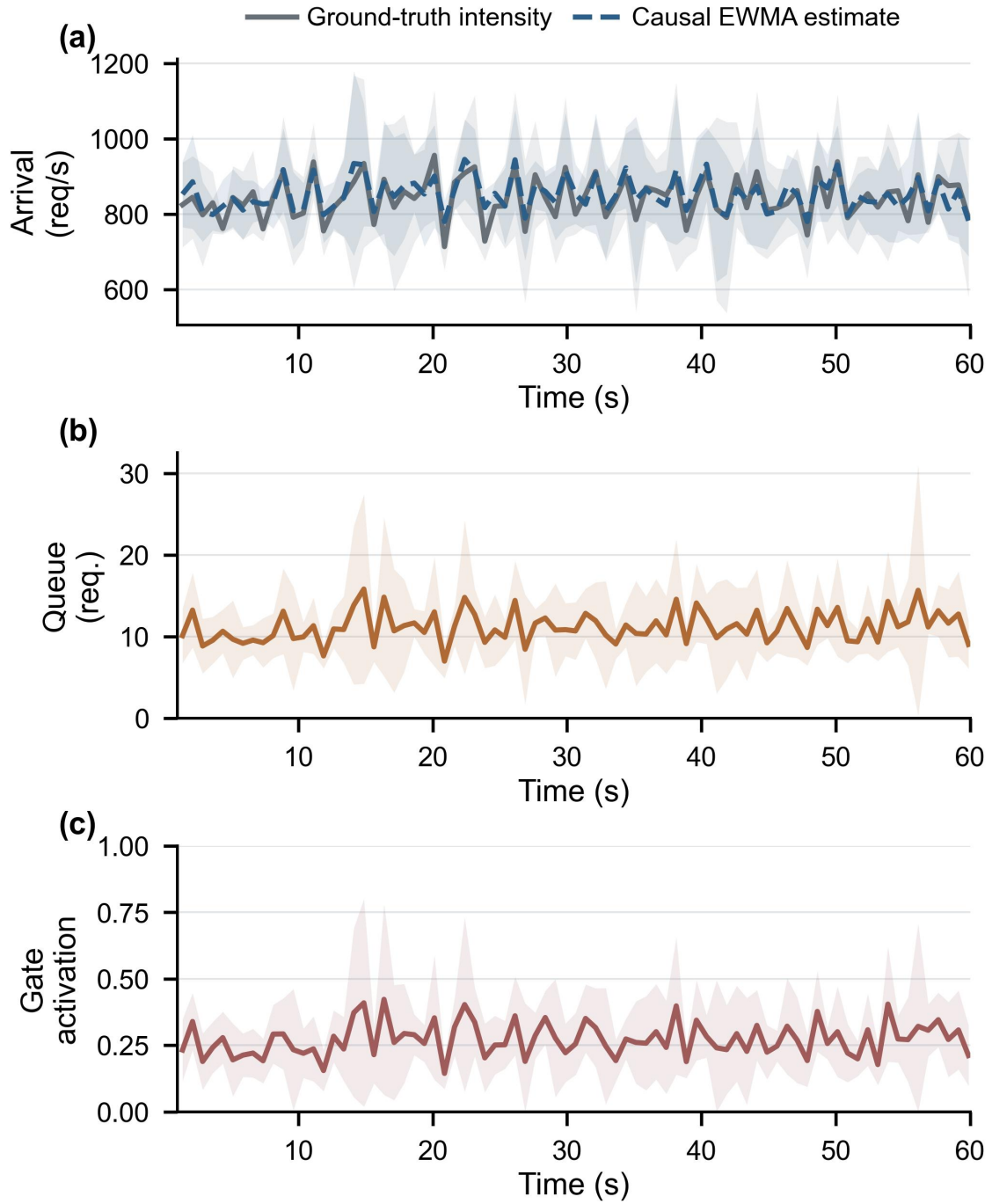


图 5.4 Random-switch 下的 Gate 机制诊断：(a) 已揭示到达的 EWMA 估计；(b) 队列长度；(c) Gate 激活率。
真实到达率只用于离线对照，不输入调度器。

到达率估计与离线真实序列的 Pearson 相关系数为 0.839，MAE 为 25.3 requests/s；队列长度与 Gate 激活率的相关系数为 0.929。低队列四分位区间的平均激活率为 0.200，高队列四分位区间增至 0.355。这些结果说明 Gate 的收缩频率会随可观测压力上升，而不是固定屏蔽动作。相关性不能证明 Gate 给出确定性安全保证，也不能证明每次筛选都正确，但它提供了与设计一致的内部行为证据。

5.5 消融实验

我们在相同 60 s 主协议下比较完整方法、去掉 Historical Feedback GRU、去掉 Causal Gate、同时去掉二者以及 Plain SAC。图 5.5 使用 publication-safe 口径展示联合消融；表 5.7 给出可直接复算的宏平均。

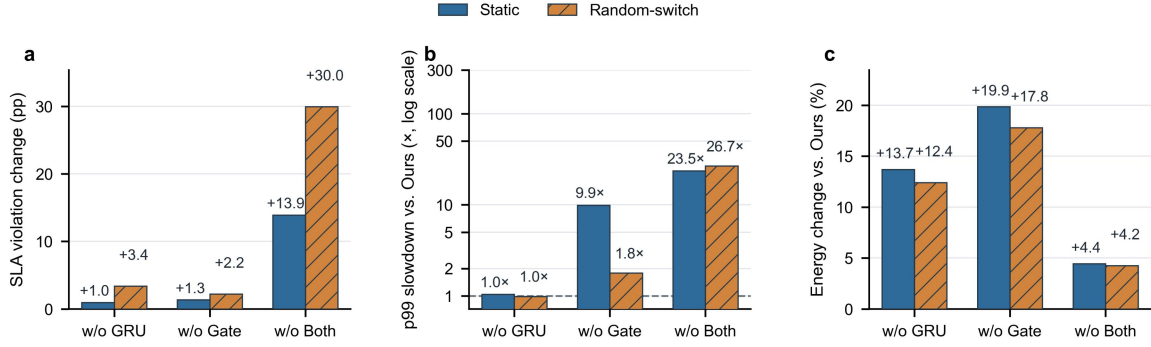


图 5.5 Historical Feedback GRU 与 Causal Gate 的联合消融。图中仅比较同一主协议下的完整方法及对应去除版本。

表 5.7 核心模块消融结果

场景	方法	违约率 (%)	p99 (ms)	能耗 (J/request)
静态	Full	33.75	92.68	0.1019
静态	w/o History-GRU	34.72	96.34	0.1158
静态	w/o Causal Gate	32.10	9316.69	0.1221
静态	w/o Both	47.66	24181.93	0.1064
静态	Plain SAC	25.07	27466.02	0.0988
动态	Full	31.32	95.84	0.0987
动态	w/o History-GRU	31.70	94.62	0.1110
动态	w/o Causal Gate	28.54	170.61	0.1163
动态	w/o Both	61.28	25754.87	0.1029
动态	Plain SAC	34.25	50725.13	0.1068

去掉 History-GRU 后，静态违约率、p99 与能耗均变差；动态 p99 则从 95.84 ms 小幅改善到 94.62 ms，但能耗上升至 0.1110 J，且违约率略升。因此，当前数据不支持“GRU 单独改善每一项动态指标”的强因果结论。去掉 Gate 时，违约率数字反而略低，但静态 p99 上升到 9.32 s、动态 p99 上升到 170.61 ms，能耗也超过预算，说明只看 aggregate 违约率会掩盖严重 backlog 与长尾。二者同时去除后，违约率和 p99 均发生显著退化，表明历史表示与风险边界存在协同作用。

Plain SAC 在静态场景出现较低违约率却伴随 27.47 s 的 p99，进一步说明稀疏违约统计不足以单独评价排队系统。现有主协议消融完整覆盖 GRU、Gate 与二者联合去除；SLA dual 和 tail-risk surrogate 的证据主要来自旧协议消融与 tail-risk weight 敏感性，尚缺在当前 60 s 主协议下逐项独立

重训。因此，本文把这些机制作为完整训练目标的组成部分，不把现有结果扩大为每个子项的独立因果证明。

5.6 开销、敏感性与局限

表 5.8 调度器开销与模型规模

方法	CPU 平均决策 (ms)	CPU p99 (ms)	部署参数	Checkpoint (MiB)
RiskBudget-SAC	4.97	8.10	198,374	6.37
Coinf-Adapted	1.86	3.56	40,230	0.48
TF-Serving-style	0.35	0.95	0	0
Triton-Dynamic-Batcher	0.52	1.62	0	0
Ebird-style	0.39	0.95	0	0

RiskBudget-SAC 的 CPU 单次决策平均耗时为 4.97 ms，p99 为 8.10 ms，明显高于启发式和 Coinf-Adapted。其部署张量约 0.758 MiB，完整 checkpoint 为 6.37 MiB，训练墙钟时间约 3640 s。当前主结果请求时延未把调度器推理时间重新计入，因而该开销相对于约 35.7–47.8 ms 的模型级 deadline 并不可忽略；后续真实系统实现需要通过批量动作编码、编译和异步决策进一步压缩。

表 5.9 RiskBudget-SAC 的敏感性结果 ($\rho=0.7$ ，独立补充协议)

因素	取值	违约率 (%)	p99 (ms)	能耗 (J/request)
Deadline scale	0.8x	10.95	53.09	0.1055
Deadline scale	1.0x	1.67	46.01	0.1006
Deadline scale	1.2x	0.25	47.59	0.0967
Interference	0.0	0.04	30.68	0.0891
Interference	0.5	0.06	34.34	0.0933
Interference	1.0	1.67	46.01	0.1006
Interference	1.5	29.53	86.68	0.1072

收紧 deadline 会显著提高违约率；干扰增强则同时推高违约率、p99 和估计能耗。在 1.5 倍干扰下，方法仍优于同协议中的主要基线，但性能已经明显下降，说明其适应能力存在边界。tail-risk weight 的补充实验表明默认权重 1.0 在模型 mix 漂移、静态负载和能耗之间较均衡，但并非每个单独场景的最优值。

本文仍有四项限制：主结果仅使用一个训练种子，5 个评估种子不能证明训练稳定性；能耗来自执行 profile，不代表整机端到端实测；主环境使用随机干扰代理，硬件测量尚未作为完整方向表进入训练；基础动作 mask 的设备忙余量可能间接反映运行变化。因此，本文只对 Gate 的因果在线估计作有限主张，结论也仅适用于当前协议。